

LAPORAN TUGAS K1

IMPLEMENTASI ENKRIPSI PLAYFAIR CIPHER

KOM1314 Keamanan Informasi

A	L	N	G	E
S	H	P	U	B
C	D	F	I	K
M	O	Q	R	T
V	W	X	Y	Z

Disusun oleh:
Afdha Auliya Atiq
NIM [ISI NIM]

DEPARTEMEN ILMU KOMPUTER
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
IPB UNIVERSITY
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Algoritma kriptografi klasik adalah algoritma berbasis karakter yang dahulu dikerjakan hanya dengan pena dan kertas. Meskipun sudah tidak aman untuk kebutuhan modern, algoritma klasik tetap penting dipelajari karena menjadi dasar pemahaman konsep kriptografi, merupakan fondasi algoritma kriptografi modern, dan memperlihatkan kelemahan-kelemahan yang harus dihindari dalam perancangan sistem cipher.

Salah satu algoritma klasik yang menarik adalah *Playfair cipher*, yaitu cipher substitusi poligram yang mengenkripsi pasangan huruf (bigram), bukan huruf tunggal. Dengan mengenkripsi bigram, distribusi frekuensi huruf pada cipherteks menjadi lebih datar sehingga analisis frekuensi huruf tunggal menjadi jauh lebih sulit dibandingkan cipher abjad-tunggal seperti Caesar cipher.

Pada tugas K1 mata kuliah Keamanan Informasi ini dibuat sebuah aplikasi web yang mengimplementasikan enkripsi Playfair cipher terhadap berkas teks, beserta laporan implementasinya.

1.2 Tujuan

1. Mengimplementasikan algoritma Playfair cipher sesuai materi kuliah Kriptografi Klasik.
2. Membangun aplikasi yang menerima berkas teks asli dan kata/frasa kunci, lalu menghasilkan berkas/teks terenkripsi.
3. Menguji kebenaran implementasi terhadap contoh pada materi kuliah serta melalui pengujian otomatis.

1.3 Spesifikasi Tugas

Tabel 1. Spesifikasi aplikasi dan pemenuhannya

Komponen	Spesifikasi	Pemenuhan pada aplikasi
Masukan	Berkas teks asli	Unggah berkas <code>.txt</code> (klik atau <i>drag & drop</i>); isi berkas tampil dan dapat disunting.
Kunci	Kata/frasa kunci	Kolom kunci; bujursangkar 5×5 diperbarui secara langsung saat kunci diketik.
Keluaran	Berkas/teks terenkripsi	Cipherteks ditampilkan, dapat disalin, dan diunduh sebagai berkas <code>*-terenkripsi.txt</code> .
Keluaran	Aplikasi program	Aplikasi web statis (HTML, CSS, JavaScript) pada folder <code>app/</code> .
Keluaran	Laporan PDF dengan screenshot	Dokumen ini (BAB IV memuat screenshot program).

1.4 Ruang Lingkup

- Alfabet yang digunakan adalah 25 huruf Latin (A–Z tanpa J); huruf J diganti I.
- Karakter selain huruf (spasi, angka, tanda baca) dibuang pada tahap pra-pemrosesan, sebagaimana Playfair klasik.
- Selain enkripsi, aplikasi juga menyediakan dekripsi untuk memverifikasi bahwa cipherteks dapat dikembalikan.

BAB II

DASAR TEORI

2.1 Cipher Substitusi Poligram

Cipher substitusi poligram mensubstitusi blok huruf plainteks dengan blok huruf cipherteks, misalnya AS diganti RT. Jika panjang blok dua huruf disebut *digram* (bigram). Tujuannya adalah membuat distribusi kemunculan poligram menjadi datar sehingga menyulitkan analisis frekuensi.

2.2 Playfair Cipher

Playfair cipher ditemukan oleh Sir Charles Wheatstone dan dipromosikan oleh Baron Lyon Playfair pada tahun 1854. Kunci kriptografinya adalah 25 huruf yang disusun dalam bujursangkar 5×5 dengan menghilangkan huruf J dari abjad, sehingga jumlah kemungkinan kunci adalah $25! = 15.511.210.043.330.985.984.000.000$.

2.2.1 Pembentukan Bujursangkar Kunci

Huruf-huruf unik dari kata/frasa kunci ditulis terlebih dahulu dari kiri ke kanan dan dari atas ke bawah, kemudian dilanjutkan dengan huruf alfabet yang belum muncul. Tabel 2 memperlihatkan bujursangkar yang digunakan pada contoh materi kuliah (huruf dari kunci diberi latar biru).

Tabel 2. Bujursangkar kunci pada contoh materi kuliah (kunci ALNGESHPUB)

A	L	N	G	E
S	H	P	U	B
C	D	F	I	K
M	O	Q	R	T
V	W	X	Y	Z

2.2.2 Pra-pemrosesan Pesan

1. Ganti huruf J (bila ada) dengan I.
2. Tulis pesan dalam pasangan huruf (bigram).
3. Jangan sampai ada pasangan huruf yang sama; jika ada, sisipkan X di tengahnya.
4. Jika jumlah huruf ganjil, tambahkan huruf X di akhir.

Contoh: plainteks `temui ibu nanti malam menjadi bigram` TE MU IX IB UN AN TI MA LA MX.

2.2.3 Algoritma Enkripsi

Misalkan huruf pertama bigram berada pada baris r_1 kolom c_1 dan huruf kedua pada baris r_2 kolom c_2 (indeks 0–4):

1. **Baris sama** ($r_1 = r_2$): tiap huruf diganti huruf di kanannya secara siklik, yaitu $(r, (c + 1) \bmod 5)$.

2. **Kolom sama** ($c_1 = c_2$): tiap huruf diganti huruf di bawahnya secara siklik, yaitu $((r + 1) \bmod 5, c)$.
3. **Persegi panjang**: huruf pertama diganti huruf pada perpotongan baris huruf pertama dan kolom huruf kedua (r_1, c_2); huruf kedua diganti huruf pada titik sudut keempat (r_2, c_1).

2.2.4 Algoritma Dekripsi

Dekripsi merupakan kebalikan enkripsi: pada baris yang sama huruf diganti huruf di kirinya, $((c + 4) \bmod 5)$; pada kolom yang sama huruf diganti huruf di atasnya, $((r + 4) \bmod 5)$; aturan persegi panjang tetap sama. Terakhir, huruf X yang tidak mengandung makna (sisipan) dibuang.

2.3 Keamanan Playfair Cipher

Terdapat $26 \times 26 = 676$ bigram sehingga identifikasi bigram individual lebih sukar daripada huruf tunggal. Namun ukuran poligram hanya dua huruf, sehingga Playfair cipher tetap tidak aman: dengan tabel frekuensi pasangan huruf (misalnya TH dan HE dalam bahasa Inggris) dan cipherteks yang cukup banyak, cipher ini dapat dipecahkan melalui analisis frekuensi bigram.

BAB III

PERANCANGAN DAN IMPLEMENTASI

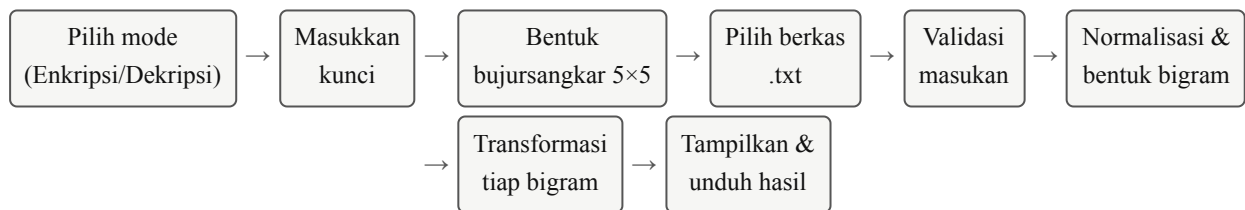
3.1 Lingkungan dan Teknologi

- **Jenis aplikasi:** aplikasi web statis; seluruh proses berjalan di peramban tanpa server dan tanpa koneksi internet.
- **Bahasa:** HTML5, CSS3, dan JavaScript (ECMAScript 2020) tanpa pustaka eksternal.
- **Pembacaan berkas:** FileReader API (UTF-8); **penyimpanan hasil:** Blob + tautan unduhan.
- **Pengujian:** Node.js v22.20.0 dengan modul bawaan `node:test`; laporan dan screenshot dibuat otomatis dengan Chrome headless (puppeteer-core).

3.2 Struktur Proyek

```
ki-k1/
├── app/
│   ├── index.html      halaman antarmuka
│   ├── style.css        tampilan
│   ├── playfair.js      modul inti algoritma Playfair (dipakai browser & Node.js)
│   └── app.js           logika antarmuka: baca berkas, proses, tampilkan, unduh
├── samples/
│   ├── plainteks.txt    berkas teks asli contoh
│   ├── plainteks-terenkripsi.txt  berkas hasil enkripsi (diunduh dari aplikasi)
│   └── plainteks-terdekripsi.txt  berkas hasil dekripsi (diunduh dari aplikasi)
├── tests/playfair.test.js  unit test
├── tools/                 skrip pembuat laporan PDF
└── laporan/              laporan HTML/PDF dan screenshot
```

3.3 Alur Program



Modul inti `playfair.js` dipisahkan dari antarmuka (`app.js`) sehingga fungsi yang sama dapat diuji langsung melalui Node.js. Setiap fungsi enkripsi/dekripsi mengembalikan hasil akhir sekaligus rincian langkah per bigram (posisi asal, aturan, posisi hasil) yang ditampilkan pada antarmuka.

3.4 Implementasi Modul Inti

3.4.1 Normalisasi dan Bujursangkar Kunci

Fungsi `normalize` mengubah teks menjadi huruf kapital, mengganti J dengan I, dan membuang karakter non-huruf. Fungsi `buildKeySquare` menggabungkan huruf kunci dengan alfabet 25 huruf, mengambil huruf unik, lalu menyusunnya ke dalam matriks 5×5 beserta tabel posisi setiap huruf agar pencarian posisi bernilai $O(1)$.

```

/** Susun bujursangkar kunci 5x5 dari kata/frasa kunci. */
function buildKeySquare(key) {
  const normalizedKey = normalize(key);
  const seen = new Set();
  const sequence = [];
  for (const ch of normalizedKey + ALPHABET) {
    if (!seen.has(ch)) {
      seen.add(ch);
      sequence.push(ch);
    }
  }

  const grid = [];
  const position = {};
  sequence.forEach((ch, i) => {
    const row = Math.floor(i / SIZE);
    const col = i % SIZE;
    if (!grid[row]) grid[row] = [];
    grid[row][col] = ch;
    position[ch] = { row, col };
  });

  return {
    key: normalizedKey,
    keyLetters: Array.from(new Set(normalizedKey)).join(''),
    sequence: sequence.join(''),
    grid,
    position,
  };
}

```

3.4.2 Pembentukan Bigram

Plainteks dibaca dua huruf sekaligus. Bila kedua huruf sama, hanya huruf pertama yang diambil dan dipasangkan dengan huruf sisipan (default X); bila tersisa satu huruf di akhir, huruf tersebut dipasangkan dengan huruf tambahan. Kasus khusus huruf kembar **XX** menggunakan sisipan Q agar tidak menghasilkan bigram kembar baru.

```

/**
 * Pecah plainteks menjadi bigram.
 * note: null (normal) | 'sisip' (huruf kembar disisipi) | 'ganjil' (tambahan di akhir)
 */
function toBigrams(text, filler) {
  const f = resolveFiller(filler);
  const s = normalize(text);
  const bigrams = [];
  let i = 0;
  while (i < s.length) {
    const a = s[i];
    const b = s[i + 1];
    if (b === undefined) {
      bigrams.push({ pair: a + fillerFor(a, f), note: 'ganjil' });
      i += 1;
    } else if (a === b) {
      bigrams.push({ pair: a + fillerFor(a, f), note: 'sisip' });
      i += 1;
    } else {
      bigrams.push({ pair: a + b, note: null });
    }
  }
}

```

```

    i += 2;
  }
}
return { normalized: s, bigrams };
}

```

3.4.3 Transformasi Bigram (Enkripsi dan Dekripsi)

Satu fungsi digunakan untuk kedua arah. Pergeseran bernilai +1 untuk enkripsi dan +4 (setara $-1 \bmod 5$) untuk dekripsi; aturan persegi panjang tidak bergantung pada arah.

```

/** Transformasi satu bigram. direction: 'encrypt' | 'decrypt'. */
function transformPair(pair, square, direction) {
  const shift = direction === 'decrypt' ? SIZE - 1 : 1;
  const p1 = square.position[pair[0]];
  const p2 = square.position[pair[1]];
  let c1;
  let c2;
  let rule;

  if (p1.row === p2.row) {
    rule = 'baris';
    c1 = { row: p1.row, col: (p1.col + shift) % SIZE };
    c2 = { row: p2.row, col: (p2.col + shift) % SIZE };
  } else if (p1.col === p2.col) {
    rule = 'kolom';
    c1 = { row: (p1.row + shift) % SIZE, col: p1.col };
    c2 = { row: (p2.row + shift) % SIZE, col: p2.col };
  } else {
    rule = 'persegi';
    c1 = { row: p1.row, col: p2.col };
    c2 = { row: p2.row, col: p1.col };
  }

  return {
    input: pair,
    output: square.grid[c1.row][c1.col] + square.grid[c2.row][c2.col],
    rule,
    from: [p1, p2],
    to: [c1, c2],
  };
}

```

3.4.4 Pembuangan Huruf Sisipan pada Dekripsi

Setelah dekripsi, X dianggap sisipan apabila berada di antara dua huruf kembar (misalnya IX IB → IIB) atau berada di akhir pesan. Karena heuristik ini dapat ikut membuang X asli, aplikasi juga menampilkan hasil mentah sebelum pembuangan.

```

/**
 * Buang huruf sisipan hasil dekripsi (langkah 4 pada slide: "buang huruf X yang tidak
 * mengandung makna"): X di antara dua huruf kembar dan X di akhir pesan.
 * Heuristik ini bisa ikut membuang X asli (mis. "EXE"), karena itu hasil mentah juga
 * disediakan.
 */
function markRemovableFillers(steps, filler) {
  steps.forEach((step, i) => {
    const p = step.output;

```



```

const next = steps[i + 1];
const isFiller = p[1] === fillerFor(p[0], filler);
step.note = null;
if (isFiller && next && next.output[0] === p[0]) step.note = 'sisip';
else if (isFiller && !next) step.note = 'ganjil';
});
return steps.map((s) => (s.note ? s.output[0] : s.output)).join('');
}

```

3.5 Fitur Antarmuka

Tabel 3. Fitur aplikasi

Fitur	Keterangan
Mode enkripsi & dekripsi	Dipilih melalui tombol segmen; label masukan dan nama berkas keluaran menyesuaikan mode.
Unggah berkas teks	Tombol pilih berkas atau <i>drag & drop</i> , batas 5 MB, isi berkas dapat disunting.
Bujursangkar kunci langsung	Matriks 5×5 diperbarui setiap kunci diketik; huruf yang berasal dari kunci ditandai.
Rincian proses	Hasil normalisasi, daftar bigram (huruf sisipan ditandai), serta tabel posisi dan aturan untuk setiap bigram; posisi disorot pada bujursangkar.
Keluaran	Format per bigram, blok 5 huruf, atau tanpa spasi; tombol Salin, Unduh .txt, dan “Gunakan sebagai masukan” untuk langsung menguji dekripsi.
Analisis frekuensi	Histogram frekuensi huruf plainteks dibandingkan cipherteks.
Validasi	Kunci kosong, teks kosong, cipherteks ganjil, dan bigram kembar pada cipherteks ditolak dengan pesan yang jelas.

BAB IV

HASIL DAN PENGUJIAN

4.1 Tampilan Program

Aplikasi dijalankan dengan membuka berkas `app/index.html` pada peramban. Halaman terdiri atas kolom utama (masukan dan hasil) serta panel samping yang menampilkan bujursangkar kunci.

The screenshot shows the 'Playfair Cipher' web application. The header includes a logo 'PF' and the title 'Playfair Cipher' with a subtitle 'Enkripsi & dekripsi berkas teks dengan bujursangkar kunci 5x5'. A user identifier 'KOM1314 Keamanan Informasi - Tugas K1' is in the top right. The main interface is divided into two columns. The left column, titled '1 Masukan', contains a 'KUNCI (KATA / FRASA)' field with the example 'KEAMANAN INFORMASI', a 'BERKAS PLAINTEKS (.TXT)' section with a file upload icon and text 'Pilih berkas atau seret & lepas di sini. Belum ada berkas dipilih', an 'ISI PLAINTEKS' text area, and a 'HURUF SISIPAN' dropdown menu set to 'X'. The 'FORMAT KELUARAN' dropdown is set to 'Per bigram (ZB RS FY ...)'. At the bottom of this column are buttons for 'Enkripsi', 'Muat contoh slide', and 'Reset', along with a keyboard shortcut 'Ctrl/⌘ + Enter'. The right column, titled 'Bujursangkar kunci', displays a 5x5 grid of letters: A, B, C, D, E; F, G, H, I, K; L, M, N, O, P; Q, R, S, T, U; V, W, X, Y, Z. Below the grid is a legend: a green square for 'Huruf bigram masukan', an orange square for 'Huruf hasil', and a grey square for 'Huruf dari kunci'. A link '► Aturan Playfair' is at the bottom of the right column. A footer note states: 'Playfair Cipher - Aplikasi web statis, seluruh proses berjalan di peramban (tanpa server)'.

Gambar 1. Tampilan awal aplikasi Playfair Cipher

4.2 Pengujian Enkripsi Berkas Teks

Pengujian menggunakan berkas `samples/plainteks.txt` dan kunci `KEAMANAN INFORMASI`. Isi berkas teks asli adalah sebagai berikut.

Keamanan informasi adalah upaya melindungi data dari akses, penggunaan, pengungkapan, dan perubahan oleh pihak yang tidak berwenang. Tiga pilar utamanya adalah kerahasiaan, integritas, dan ketersediaan. Playfair cipher ditemukan oleh Sir Charles Wheatstone pada tahun 1854 dan dipopulerkan oleh Baron Lyon Playfair. Cipher ini mengenkripsi pasangan huruf sehingga analisis frekuensi huruf tunggal menjadi jauh lebih sulit.

PF

Playfair Cipher
Enkripsi & dekripsi berkas teks dengan bujursangkar kunci 5x5

KOM1314 Keamanan Informasi - Tugas K1

1 Masukan

Enkripsi

Dekripsi

KUNCI (KATA / FRASA)

KEAMANAN INFORMASI

BERKAS PLAINTEKS (.TXT)

Pilih berkas atau seret & lepas di sini

plainteks.txt · 423 B

ISI PLAINTEKS (bisa juga diketik atau diedit langsung)

Keamanan informasi adalah upaya melindungi data dari akses, penggunaan, pengungkapan, dan perubahan oleh pihak yang tidak berwenang. Tiga pilar utamanya adalah kerahasiaan, integritas, dan ketersediaan. Playfair cipher ditemukan oleh Sir Charles Wheatstone pada tahun 1854 dan dipopulerkan oleh Baron Lyon Playfair. Cipher ini mengenkripsi pasangan huruf sehingga analisis frekuensi huruf tunggal menjadi jauh lebih sulit.

HURUF SISIPAN

X

Per bigram (ZB RS FY ...)

Enkripsi

Muat contoh slide

Reset

Ctrl/⌘ + Enter

Bujursangkar kunci

K	E	A	M	N
I	F	O	R	S
B	C	D	G	H
L	P	Q	T	U
V	W	X	Y	Z

Huruf unik dari kunci: KEAMNIFORS

Huruf bigram masukan

Huruf hasil

Huruf dari kunci

► Aturan Playfair

Playfair Cipher - Aplikasi web statis, seluruh proses berjalan di peramban (tanpa server).

Gambar 2. Masukan enkripsi: kunci "KEAMANAN INFORMASI" dan berkas plainteks.txt yang telah dipilih

Setelah kunci diketik, panel samping langsung menampilkan bujursangkar kunci. Berkas yang dipilih ditampilkan nama dan ukurannya, dan isinya dimuat ke kolom teks.

Tabel 4. Bujursangkar kunci "KEAMANAN INFORMASI"

K	E	A	M	N
I	F	O	R	S
B	C	D	G	H
L	P	Q	T	U
V	W	X	Y	Z

2 Cipherteks

Salin Unduh .txt

Gunakan sebagai masukan ⇌

355

179

3

47/28/104

huruf plaintext

bigram

huruf sisipan ditambah

baris / kolom / persegi

EA MN MK MK SK OR GR NO OK QO QK UZ QE XM NA VB AH ZS BR QO

QM QO SF ME FN FU AK DY HT KM MK WF MH ZS BM EQ MK QO EU MF

LH ND MK IQ NC LF DN MV MK TY OB ME CK FY AK MK TY RB EQ BV

MO LU MN MK XM OQ KQ ND EA OM DN IF OA MK SK PM TG RL NO QO

KE MP MF FN BO OA MK QP MX OE FS BF UC MF BO PM NT EM AS PK

UH FS DB MO PK FZ CN MQ RU SA FW OQ MQ ND ZS QO AH FL FO LP

ME FN AS BK AS NO SA TH CA OB MX OE FS BF UC ME CK BK AK CM

3 Pra-pemrosesan

1. Normalisasi huruf kapital, 2 huruf J → I, 68 karakter non-huruf di buang

KEAMANANINFORMASIADALAHUPAYAMELINDUNGIDATADARIAKSESPENGGUNAANPENGUNGKAPANPERUBAHANOLEHPIHA

KYANGTIDAKBERWENANGTIGAPILARUTAMANYAADALAHKERAHASIAANINTEGRITASDANKETERSEDIAANPLAYFAIRCIPHERD

ITEMUKANOLEHSIRCHARLESWHEATSTONEPADATAHUNDANDIPOPULERKANOLEHBARONLYONPLAYFAIRCIPHERINIMENGK

RIPSIPASANGANHURUFSEHINGGAANALISISFREKUENSIHURUFTUNGALMENIADIIAUHLEBIHSULIT

2. Bentuk bigram huruf bergaris bawah = sisipan "X" (huruf kembar / jumlah ganjil)

KE AM AN AN IN FO RM AS IA DA LA HU PA YA ME LI ND UN GI DA

TA DA RI AK SE SP EN GX GU NA AN PE NG UN GK AP AN DA NP ER

UB AH AN OL EH PI HA KY AN GT ID AK BE RW EN AN GT IG AP IL

AR UT AM AN YA AD AL AH KE RA HA SI AX AN IN TE GR IT AS DA

NK ET ER SE DI AX AN PL AY FA IR CI PH ER DI TE MU KA NO LE

HS IR CH AR LE SW HE AT ST ON EP AD AT AH UN DA ND IP OP UL

ER KA NO LE HB AR ON LY ON PL AY FA IR CI PH ER IN IM EN GE

4 Proses per bigram

Arahkan kursor ke baris untuk melihat posisinya di bujursangkar

NO	BIGRAM	POSISI (B,K)	ATURAN	POSISI HASIL	HASIL	KETERANGAN
----	--------	--------------	--------	--------------	-------	------------

Bujursangkar kunci

K	E	A	M	N
I	F	O	R	S
B	C	D	G	H
L	P	Q	T	U
V	W	X	Y	Z

KE → EA baris

Huruf unik dari kunci: KEAMNIFORS

Huruf bigram masukan

Huruf hasil

Huruf dari kunci

► Aturan Playfair

Gambar 3. Hasil enkripsi: cipherteks, statistik proses, serta tombol Salin dan Unduh .txt

Proses enkripsi menghasilkan 179 bigram dari 355 huruf, dengan 3 huruf sisipan. Distribusi aturan: 47 baris sama, 28 kolom sama, dan 104 persegi panjang.

3 Pra-pemrosesan

1. Normalisasi huruf kapital, 2 huruf J → I, 68 karakter non-huruf di buang

KEAMANANINFORMASIADALAHUPAYAMELINDUNGIDATADARIAKSESPENGGUNAANPENGUNGKAPANPERUBAHANOLEHPIHA

KYANGTIDAKBERWENANGTIGAPILARUTAMANYAADALAHKERAHASIAANINTEGRITASDANKETERSEDIAANPLAYFAIRCIPHERD

ITEMUKANOLEHSIRCHARLESWHEATSTONEPADATAHUNDANDIPOPULERKANOLEHBARONLYONPLAYFAIRCIPHERINIMENGK

RIPSIPASANGANHURUFSEHINGGAANALISISFREKUENSIHURUFTUNGALMENIADIIAUHLEBIHSULIT

2. Bentuk bigram huruf bergaris bawah = sisipan "X" (huruf kembar / jumlah ganjil)

KE AM AN AN IN FO RM AS IA DA LA HU PA YA ME LI ND UN GI DA

TA DA RI AK SE SP EN GX GU NA AN PE NG UN GK AP AN DA NP ER

UB AH AN OL EH PI HA KY AN GT ID AK BE RW EN AN GT IG AP IL

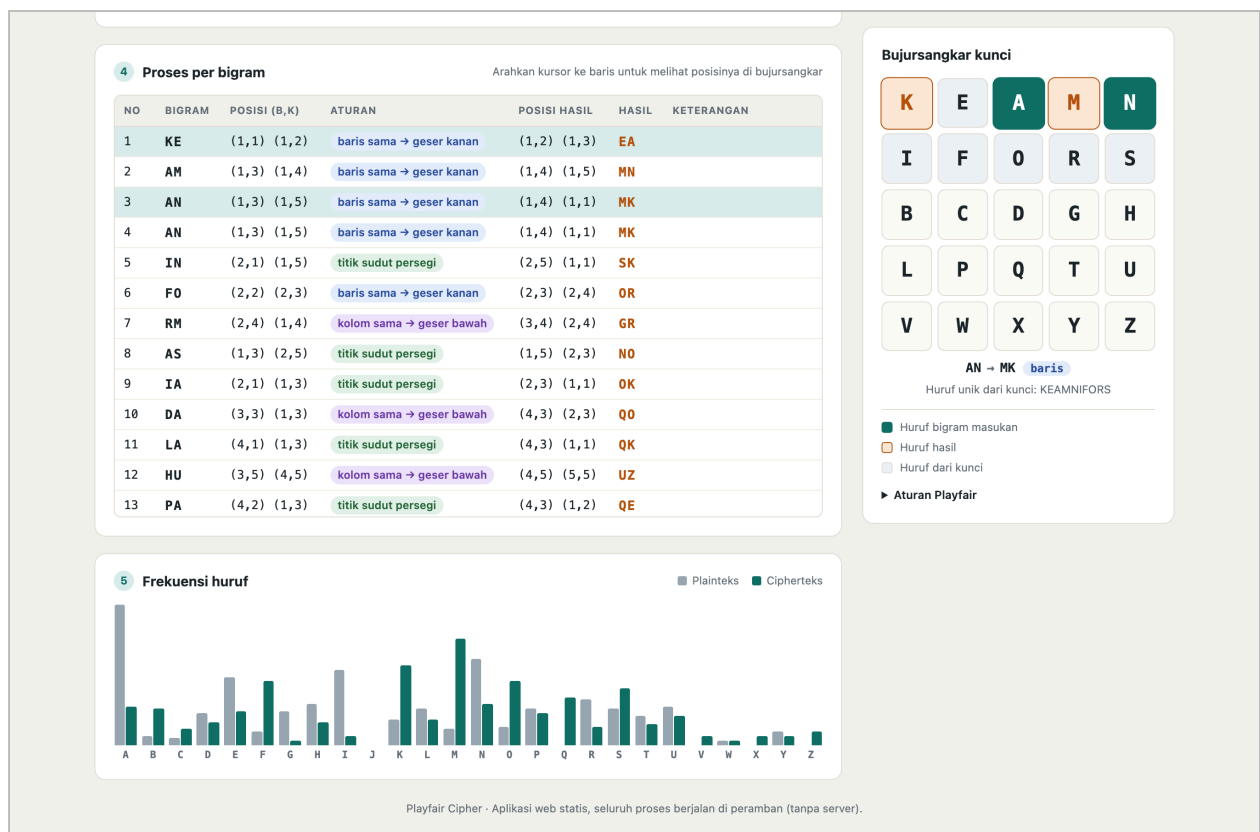
AR UT AM AN YA AD AL AH KE RA HA SI AX AN IN TE GR IT AS DA

NK ET ER SE DI AX AN PL AY FA IR CI PH ER DI TE MU KA NO LE

HS IR CH AR LE SW HE AT ST ON EP AD AT AH UN DA ND IP OP UL

ER KA NO LE HB AR ON LY ON PL AY FA IR CI PH ER IN IM EN GE

Gambar 4. Tahap pra-pemrosesan: normalisasi teks dan pembentukan bigram (huruf sisipan ditandai)



Gambar 5. Tabel proses per bigram; baris yang disorot ditampilkan posisinya pada bujursangkar kunci

Tabel 5. Dua belas langkah pertama enkripsi berkas contoh

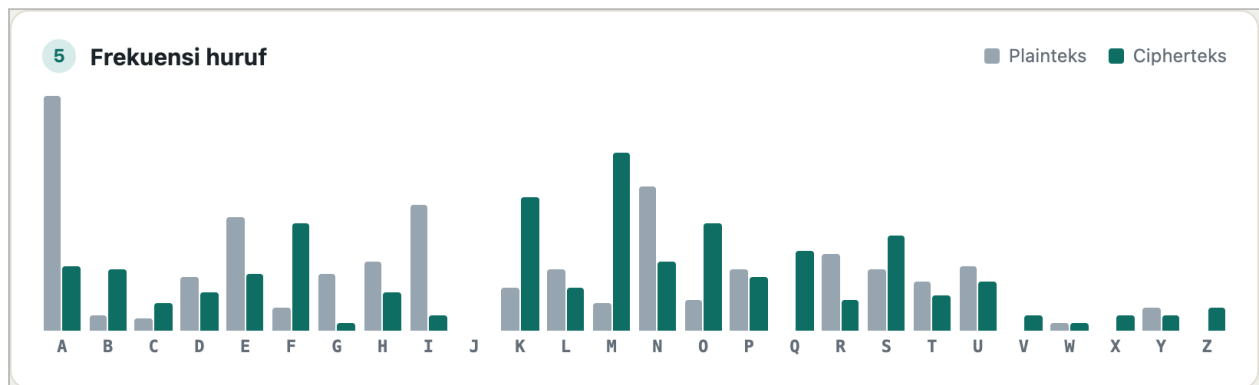
No	Bigram	Posisi (b,k)	Aturan	Posisi hasil	Hasil
1	KE	(1,1) (1,2)	Baris sama	(1,2) (1,3)	EA
2	AM	(1,3) (1,4)	Baris sama	(1,4) (1,5)	MN
3	AN	(1,3) (1,5)	Baris sama	(1,4) (1,1)	MK
4	AN	(1,3) (1,5)	Baris sama	(1,4) (1,1)	MK
5	IN	(2,1) (1,5)	Persegi panjang	(2,5) (1,1)	SK
6	FO	(2,2) (2,3)	Baris sama	(2,3) (2,4)	OR
7	RM	(2,4) (1,4)	Kolom sama	(3,4) (2,4)	GR
8	AS	(1,3) (2,5)	Persegi panjang	(1,5) (2,3)	NO
9	IA	(2,1) (1,3)	Persegi panjang	(2,3) (1,1)	OK
10	DA	(3,3) (1,3)	Kolom sama	(4,3) (2,3)	QO
11	LA	(4,1) (1,3)	Persegi panjang	(4,3) (1,1)	QK
12	HU	(3,5) (4,5)	Kolom sama	(4,5) (5,5)	UZ

Huruf sisipan ditambahkan pada bigram: ke-28 GX (sisipan (huruf kembar)), ke-73 AX (sisipan (huruf kembar)), ke-86 AX (sisipan (huruf kembar)).

Berkas hasil enkripsi yang diunduh dari aplikasi (`samples/plainteks-terenripsi.txt`) berisi cipherteks berikut.

```
EA MN MK MK SK OR GR NO OK QO QK UZ QE XM NA VB AH ZS BR QO
QM QO SF ME FN FU AK DY HT KM MK WF MH ZS BM EQ MK QO EU MF
LH ND MK IQ NC LF DN MV MK TY OB ME CK FY AK MK TY RB EQ BV
MO LU MN MK XM OQ KQ ND EA OM DN IF OA MK SK PM TG RL NO QO
KE MP MF FN BO OA MK QP MX OE FS BF UC MF BO PM NT EM AS PK
UH FS DB MO PK FZ CN MQ RU SA FW OQ MQ ND ZS QO AH FL FQ LP
MF EM AS PK BC MO SA TV SA QP MX OE FS BF UC MF SK RK AK CM
KE SF UF FL NO MK DM SU TS PS FN BS MH DM MK KQ FI FI OS AE
PN SH SB TS PS UL MH DM TK AK OK BO OK ZU PK LB UH LP RL
```

Isi berkas unduhan dibandingkan dengan keluaran fungsi `encrypt` yang dijalankan terpisah di Node.js: **identik.**



Gambar 6. Perbandingan frekuensi huruf plainteks dan cipherteks

Nilai *index of coincidence* (IC) plainteks adalah 0,0766 sedangkan cipherteks 0,0610. Huruf terbanyak pada plainteks adalah A (17,5%), N (10,7%), I (9,3%), E (8,5%), R (5,6%); pada cipherteks M (13,1%), K (9,8%), F (7,8%), O (7,8%), S (7,0%). IC cipherteks yang lebih rendah menunjukkan distribusi huruf cipherteks lebih datar, sesuai tujuan Playfair cipher untuk menyamarkan frekuensi huruf tunggal. Meskipun demikian, frekuensi bigram tetap terjaga sehingga cipher masih rentan terhadap analisis frekuensi pasangan huruf.

4.3 Pengujian Dekripsi

Untuk memastikan cipherteks dapat dikembalikan, berkas `samples/plainteks-terenkripsi.txt` didekripsi kembali dengan kunci yang sama.

KEAMANAN INFORMASI ADALAH UPAYA MELINDUNGKAN DATA DARI AKSES PENGGUNAAN PENGUNGKAPAN DAN PERUBAHAN OLEH PIHAK YANG TIDAK BERWENANG. TIGA PILAR UTAMANYA ADALAH KERAHASIAAN, INTEGRITAS, DAN KETERSEDIAAN. PLAYFAIR CIPHER DITEMUKAN OLEH SIR CHARLES WHEATSTONE PADA TAHUN 1818 DAN DIPOPULERKAN OLEH BARON VON PLAYFAIR. CIPHER INI MENGENAKAN KRIPSIPASANGAN HURUF SEHINGGA ANALISIS FREKUENSI HURUF TUNGGALEMENJADI SANGAT SULIT.

Tabel 6. Hasil verifikasi dekripsi

Pemeriksaan	Hasil
Hasil dekripsi mentah sama dengan rangkaian bigram plainteks (termasuk huruf sisipan)	Sesuai
Hasil dekripsi setelah huruf sisipan dibuang sama dengan plainteks ternormalisasi	Sesuai

Perlu dicatat bahwa hasil dekripsi tidak memuat spasi, tanda baca, dan huruf J, karena informasi tersebut memang dihilangkan pada tahap pra-pemrosesan Playfair cipher.

4.4 Verifikasi terhadap Contoh Materi Kuliah

Implementasi diuji menggunakan contoh pada materi kuliah: plainteks `temui ibu nanti malam` dengan bujursangkar pada Tabel 2. Tombol **Muat contoh slide** pada aplikasi memuat contoh ini secara otomatis.

1 Masukan

Enkripsi Dekripsi

KUNCI (KATA / FRASA)

ALNGESHPUB

BERKAS PLAINTeKS (.TXT)

Pilih berkas atau seret & lepas di sini

contoh-slide.txt

ISI PLAINTeKS (bisa juga diketik atau diedit langsung)

temui ibu nanti malam

HURUF SISIPAN

FORMAT KELUARAN

X

Per bigram (ZB RS FY ...)

Enkripsi

Muat contoh slide

Reset

Ctrl/⌘ + Enter

2 Cipherteks

Salin Unduh .txt Gunakan sebagai masukan ⇄

18

huruf plainteks

10

bigram

2

huruf sisipan ditambah

2/2/6

baris / kolom / persegi

ZB RS FY KU PG LG RK VS NL QV

Bujursangkar kunci

A	L	N	G	E
S	H	P	U	B
C	D	F	I	K
M	O	Q	R	T
V	W	X	Y	Z

TE → ZB kolom

Huruf unik dari kunci: ALNGESHPUB

Huruf bigram masukan

Huruf hasil

Huruf dari kunci

▶ Aturan Playfair

Gambar 9. Verifikasi contoh slide: "temui ibu nanti malam" menghasilkan ZB RS FY KU PG LG RK VS NL QV

Tabel 7. Perbandingan hasil aplikasi dengan contoh materi kuliah

No	Bigram	Aturan	Hasil aplikasi	Hasil pada slide	Status
1	TE	Kolom sama	ZB	ZB	✓
2	MU	Persegi panjang	RS	RS	✓
3	IX	Persegi panjang	FY	FY	✓
4	IB	Persegi panjang	KU	KU	✓
5	UN	Persegi panjang	PG	PG	✓
6	AN	Baris sama	LG	LG	✓
7	TI	Persegi panjang	RK	RK	✓
8	MA	Kolom sama	VS	VS	✓
9	LA	Baris sama	NL	NL	✓
10	MX	Persegi panjang	QV	QV	✓

Cipherteks aplikasi ZB RS FY KU PG LG RK VS NL QV **sama persis** dengan cipherteks pada slide ZB RS FY KU PG LG RK VS NL QV . Contoh aturan tunggal pada slide (DI→FK, QT→RM, NQ→PX, OW→WL, HZ→BW) juga diuji pada unit test.

4.5 Validasi Masukan

1 Masukan

Enkripsi Dekripsi

KUNCI (KATA / FRASA)

KUNCI

BERKAS CIPHERTEKS (.TXT)

Pilih berkas atau seret & lepas di sini
Belum ada berkas dipilih

ISI CIPHERTEKS (bisa juga diketik atau diedit langsung)

ABCDE

HURUF SISIPAN FORMAT KELUARAN

X Tanpa spasi

Panjang cipherteks (5 huruf) ganjil. Cipherteks Playfair selalu berjumlah genap.

Dekripsi Muat contoh slide Reset

Ctrl/⌘ + Enter

Gambar 10. Validasi masukan: cipherteks berjumlah ganjil ditolak dengan pesan kesalahan

Aplikasi menolak cipherteks dengan jumlah huruf ganjil karena cipherteks Playfair selalu berjumlah genap. Validasi serupa berlaku untuk kunci kosong, teks kosong, dan bigram kembar pada cipherteks.

4.6 Pengujian Otomatis (Unit Test)

Pengujian otomatis dijalankan dengan perintah `npm test` (Node.js v22.20.0). Hasil: **10 dari 10 kasus lulus**.

Tabel 8. Daftar kasus unit test

No	Kasus uji	Status
1	bujursangkar kunci sesuai contoh slide 68	Lulus
2	bujursangkar tanpa huruf J, huruf unik, J pada kunci menjadi I	Lulus
3	bigram contoh slide 67: "temui ibu nanti malam"	Lulus
4	aturan per bigram sesuai slide 68–70	Lulus
5	enkripsi contoh slide 71	Lulus
6	dekripsi contoh slide mengembalikan plainteks dan membuang X sisipan	Lulus
7	normalisasi: J -> I, diakritik dan non-huruf dibuang	Lulus
8	huruf sisipan X pada "XX" diganti Q	Lulus
9	dekripsi menolak cipherteks ganjil atau berisi bigram kembar	Lulus
10	round-trip acak: $\text{dekripsi}(\text{enkripsi}(p)) == \text{bigram plainteks}$	Lulus

BAB V

KESIMPULAN

1. Playfair cipher berhasil diimplementasikan sebagai aplikasi web yang menerima berkas teks asli dan kata/frasa kunci, lalu menghasilkan cipherteks yang dapat disalin maupun diunduh sebagai berkas teks.
2. Implementasi mengikuti aturan pada materi kuliah: bujursangkar 5×5 tanpa J, penyisipan X pada huruf kembar dan jumlah ganjil, serta aturan baris, kolom, dan persegi panjang. Hasil enkripsi contoh `temui ibu nanti malam` sama persis dengan slide (`ZB RS FY KU PG LG RK VS NL QV`).
3. Dekripsi berkas hasil enkripsi mengembalikan plainteks ternormalisasi, dan 10 dari 10 unit test (termasuk 500 pengujian *round-trip* acak) lulus.
4. Rincian proses per bigram dan sorotan pada bujursangkar kunci membantu memahami cara kerja algoritma. Analisis frekuensi memperlihatkan bahwa Playfair menyamarkan frekuensi huruf tunggal, tetapi karena blok hanya dua huruf, cipher ini tetap rentan terhadap analisis frekuensi bigram dan tidak layak dipakai untuk pengamanan data modern.

Lampiran A. Cara Menjalankan Aplikasi

1. Buka berkas `app/index.html` menggunakan peramban modern (Chrome, Edge, Firefox, atau Safari). Tidak diperlukan instalasi maupun server.
2. Pilih mode **Enkripsi**, ketik kunci, lalu pilih atau seret berkas `.txt`.
3. Klik **Enkripsi** (atau `Ctrl/⌘ + Enter`), kemudian klik **Unduh .txt** untuk menyimpan berkas terenkripsi.
4. Untuk dekripsi, pilih mode **Dekripsi**, gunakan kunci yang sama, dan pilih berkas cipherteks.
5. Unit test: `npm test`. Membuat ulang laporan: `npm install` lalu `npm run report`.

Lampiran B. Kode Sumber Modul Inti (`app/playfair.js`)

```
/**
 * playfair.js – Implementasi inti Playfair Cipher.
 *
 * Aturan mengikuti materi K2 Kriptografi Klasik (KOM1314):
 * - Bujursangkar kunci 5x5 berisi 25 huruf (huruf J dihilangkan, J -> I).
 * - Pesan ditulis dalam bigram; huruf kembar dalam satu bigram disisipi X,
 *   dan jika jumlah huruf ganjil ditambahkan X di akhir.
 * - Enkripsi: baris sama -> geser kanan, kolom sama -> geser bawah,
 *   selain itu -> titik sudut persegi panjang.
 * - Dekripsi: kebalikannya (geser kiri / geser atas), lalu buang X sisipan.
 *
 * Modul ini dapat dipakai di browser (window.Playfair) maupun Node.js (require).
 */
(function (root, factory) {
  if (typeof module === 'object' && module.exports) {
    module.exports = factory();
  } else {
    root.Playfair = factory();
  }
})(this, function () {
```

```

}
})(typeof self !== 'undefined' ? self : this, function () {
  'use strict';

  const SIZE = 5;
  const ALPHABET = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'; // 25 huruf, tanpa J

  /** Kapitalkan, hapus diakritik, ganti J -> I, dan buang semua karakter non-huruf. */
  function normalize(text) {
    return String(text == null ? '' : text)
      .normalize('NFD')
      .replace(/[~]/g, '')
      .toUpperCase()
      .replace(/J/g, 'I')
      .replace(/[^A-Z]/g, '');
  }

  /** Susun bujursangkar kunci 5x5 dari kata/frasa kunci. */
  function buildKeySquare(key) {
    const normalizedKey = normalize(key);
    const seen = new Set();
    const sequence = [];
    for (const ch of normalizedKey + ALPHABET) {
      if (!seen.has(ch)) {
        seen.add(ch);
        sequence.push(ch);
      }
    }

    const grid = [];
    const position = {};
    sequence.forEach((ch, i) => {
      const row = Math.floor(i / SIZE);
      const col = i % SIZE;
      if (!grid[row]) grid[row] = [];
      grid[row][col] = ch;
      position[ch] = { row, col };
    });

    return {
      key: normalizedKey,
      keyLetters: Array.from(new Set(normalizedKey)).join(''),
      sequence: sequence.join(''),
      grid,
      position,
    };
  }

  /** Huruf sisipan; jika sama dengan huruf yang dipisahkan (mis. "XX"), pakai Q (atau Z). */
  function fillerFor(letter, filler) {
    if (letter !== filler) return filler;
    return filler === 'Q' ? 'Z' : 'Q';
  }

  function resolveFiller(filler) {
    return normalize(filler).charAt(0) || 'X';
  }

  /**
   * Pecah plainteks menjadi bigram.

```

```

    * note: null (normal) | 'sisip' (huruf kembar disisipi) | 'ganjil' (tambahan di akhir)
    */
function toBigrams(text, filler) {
    const f = resolveFiller(filler);
    const s = normalize(text);
    const bigrams = [];
    let i = 0;
    while (i < s.length) {
        const a = s[i];
        const b = s[i + 1];
        if (b === undefined) {
            bigrams.push({ pair: a + fillerFor(a, f), note: 'ganjil' });
            i += 1;
        } else if (a === b) {
            bigrams.push({ pair: a + fillerFor(a, f), note: 'sisip' });
            i += 1;
        } else {
            bigrams.push({ pair: a + b, note: null });
            i += 2;
        }
    }
    return { normalized: s, bigrams };
}

/** Transformasi satu bigram. direction: 'encrypt' | 'decrypt'. */
function transformPair(pair, square, direction) {
    const shift = direction === 'decrypt' ? SIZE - 1 : 1;
    const p1 = square.position[pair[0]];
    const p2 = square.position[pair[1]];
    let c1;
    let c2;
    let rule;

    if (p1.row === p2.row) {
        rule = 'baris';
        c1 = { row: p1.row, col: (p1.col + shift) % SIZE };
        c2 = { row: p2.row, col: (p2.col + shift) % SIZE };
    } else if (p1.col === p2.col) {
        rule = 'kolom';
        c1 = { row: (p1.row + shift) % SIZE, col: p1.col };
        c2 = { row: (p2.row + shift) % SIZE, col: p2.col };
    } else {
        rule = 'persegi';
        c1 = { row: p1.row, col: p2.col };
        c2 = { row: p2.row, col: p1.col };
    }

    return {
        input: pair,
        output: square.grid[c1.row][c1.col] + square.grid[c2.row][c2.col],
        rule,
        from: [p1, p2],
        to: [c1, c2],
    };
}

function countRules(steps) {
    const rules = { baris: 0, kolom: 0, persegi: 0 };
    steps.forEach((s) => { rules[s.rule] += 1; });
    return rules;
}

```

```

function encrypt(plaintext, key, options) {
  const opts = options || {};
  const filler = resolveFiller(opts.filler);
  const square = buildKeySquare(key);
  const { normalized, bigrams } = toBigrams(plaintext, filler);
  const steps = bigrams.map((b) =>
    Object.assign(transformPair(b.pair, square, 'encrypt'), { note: b.note })
  );
  return {
    mode: 'encrypt',
    filler,
    square,
    normalized,
    steps,
    result: steps.map((s) => s.output).join(''),
    stats: {
      letters: normalized.length,
      bigrams: steps.length,
      fillers: steps.filter((s) => s.note).length,
      rules: countRules(steps),
    },
  };
}

/**
 * Buang huruf sisipan hasil dekripsi (langkah 4 pada slide: "buang huruf X yang tidak
 * mengandung makna"): X di antara dua huruf kembar dan X di akhir pesan.
 * Heuristik ini bisa ikut membuang X asli (mis. "EXE"), karena itu hasil mentah juga
 * disediakan.
 */
function markRemovableFillers(steps, filler) {
  steps.forEach((step, i) => {
    const p = step.output;
    const next = steps[i + 1];
    const isFiller = p[1] === fillerFor(p[0], filler);
    step.note = null;
    if (isFiller && next && next.output[0] === p[0]) step.note = 'sisip';
    else if (isFiller && !next) step.note = 'ganjil';
  });
  return steps.map((s) => (s.note ? s.output[0] : s.output)).join('');
}

function decrypt(ciphertext, key, options) {
  const opts = options || {};
  const filler = resolveFiller(opts.filler);
  const square = buildKeySquare(key);
  const normalized = normalize(ciphertext);

  if (normalized.length % 2 !== 0) {
    throw new Error(
      `Panjang ciphertexts (${normalized.length} huruf) ganjil. Ciphertexts Playfair
      selalu berjumlah genap.`
    );
  }

  const steps = [];
  for (let i = 0; i < normalized.length; i += 2) {
    const pair = normalized.slice(i, i + 2);
    if (pair[0] === pair[1]) {
      throw new Error(

```

`Bigram ke- $\{i / 2 + 1\}$  "\${pair}" berisi huruf kembar, sehingga bukan cipherteks Playfair yang valid.`

```
    );
  }
  steps.push(transformPair(pair, square, 'decrypt'));
}

const rawResult = steps.map((s) => s.output).join('');
const result = markRemovableFillers(steps, filler);
return {
  mode: 'decrypt',
  filler,
  square,
  normalized,
  steps,
  rawResult,
  result,
  stats: {
    letters: normalized.length,
    bigrams: steps.length,
    fillers: steps.filter((s) => s.note).length,
    rules: countRules(steps),
  },
};
}

/** Format keluaran: groupSize 2 (bigram), 5 (blok), atau 0 (tanpa spasi). */
function format(text, groupSize, groupsPerLine) {
  const size = Number(groupSize) || 0;
  if (!size) return text;
  const groups = text.match(new RegExp(`.{1,${size}}`, 'g')) || [];
  const perLine = groupsPerLine || (size === 2 ? 20 : 12);
  const lines = [];
  for (let i = 0; i < groups.length; i += perLine) {
    lines.push(groups.slice(i, i + perLine).join(' '));
  }
  return lines.join('\n');
}

/** Frekuensi kemunculan huruf A-Z. */
function letterFrequency(text) {
  const counts = new Array(26).fill(0);
  for (const ch of String(text).toUpperCase()) {
    const code = ch.charCodeAt(0) - 65;
    if (code >= 0 && code < 26) counts[code] += 1;
  }
  return counts;
}

return {
  SIZE,
  ALPHABET,
  normalize,
  buildKeySquare,
  toBigrams,
  transformPair,
  encrypt,
  decrypt,
  format,
  letterFrequency,
```



```
};  
});
```